

## Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Phase 1: Data Collection and Tokenizer Construction</b>       | <b>1</b>  |
| <b>2</b> | <b>Phase 2: Model Implementation, Pretraining and Evaluation</b> | <b>3</b>  |
| <b>3</b> | <b>Phase 3: Reasoning Finetuning and Attention Analysis</b>      | <b>5</b>  |
| <b>4</b> | <b>Required Questions</b>                                        | <b>8</b>  |
| <b>5</b> | <b>Bonus: Retraining Without Positional Embeddings</b>           | <b>9</b>  |
| <b>6</b> | <b>Limitations</b>                                               | <b>9</b>  |
| <b>7</b> | <b>Where Automation Substituted for Human Judgment</b>           | <b>10</b> |

## List of Figures

|    |                                                                   |    |
|----|-------------------------------------------------------------------|----|
| 1  | Corpus composition by collection type . . . . .                   | 2  |
| 2  | Split composition; fertility against vocabulary size . . . . .    | 3  |
| 3  | Pretraining on identical 500M-token budgets . . . . .             | 4  |
| 4  | Marathi attention heatmaps, layers 0 and 6 . . . . .              | 5  |
| 5  | Attention entropy and distance by layer . . . . .                 | 5  |
| 6  | Phase 3 calibration sweeps: learning rate, size, epochs . . . . . | 6  |
| 7  | Phase 3 accuracy against both floors . . . . .                    | 7  |
| 8  | Error taxonomy of every parseable prediction . . . . .            | 7  |
| 9  | Answer-position attention on the entity names . . . . .           | 8  |
| 10 | Bonus: control against ablated training curves . . . . .          | 9  |
| 11 | Bonus: layer 0 with and without positional embeddings . . . . .   | 10 |

## List of Tables

|   |                                                       |   |
|---|-------------------------------------------------------|---|
| 1 | Phase 1 corpus and tokenizer statistics . . . . .     | 2 |
| 2 | Architecture and training budget . . . . .            | 3 |
| 3 | Phase 2 intrinsic and generation evaluation . . . . . | 4 |
| 4 | Phase 3 results against both floors . . . . .         | 7 |

# Language Models and Agents: Individual Project

## Building and Analysing Two Monolingual Transformer Language Models in Marathi and Konkani

Final Report: Phases 1–3

Prakash Bhabad

Two decoder-only Transformers were built from scratch, one per language. Model H is Marathi, Model L is Konkani in Devanagari. They share no corpus, no tokenizer, no vocabulary and no weights at any point.

What each phase produced:

- **Phase 1.** 389.2M Marathi and 266.2M Konkani words collected and cleaned; two independent 2,500-piece SentencePiece BPE tokenizers.
- **Phase 2.** Two 24,892,356-parameter models, built from primitives and pretrained from scratch on 499,908,608 tokens each.
- **Phase 3.** A five-family synthetic comparative-reasoning benchmark per language, supervised finetuning with a prompt-masked loss, evaluation against two floors, a per-prediction error taxonomy, attention re-measured on reasoning prompts.
- **Bonus.** A Marathi retrain with positional embeddings removed, on its own branch.

### 1. Phase 1: Data Collection and Tokenizer Construction

Marathi is Model H because both halves of the requirement were reachable. IndicCorp v2 alone ships roughly 27.8M CC-0 documents, and Marathi news publishers keep deep sitemap-indexed archives that can be crawled and cleaned by hand.

Konkani is Model L and is genuinely scarce. Sangraha, the largest systematic Indic corpus effort, holds 14,491 Konkani rows against millions for Marathi. The two languages are closely related and share a script, which made the independence requirement harder, not easier. Published corpora mislabel one as the other, and no script check can separate them.

The corpus is Devanagari only. Romi and Kannada-script Konkani were both excluded, because Devanagari is the official script in Goa and because a 25M-parameter model has limited capacity for two orthographies competing for one embedding budget. The cost is recorded: 4,651 rows, 32% of Sangraha’s `gom` split, were rejected as correctly-labelled Konkani in an out-of-scope script.

**Cleaning pipeline**, applied per document by the collector that produced it:

- (1) Unicode NFC normalization, preserving ZWJ and ZWNJ. Both are meaningful in Devanagari and stripping them alters words silently.
- (2) Whitespace normalization and OCR line-fragment repair.
- (3) Script gate: at least 70% Devanagari by non-whitespace character.
- (4) Length gate: minimum 25 words.
- (5) Marathi/Konkani language discrimination.
- (6) Exact deduplication by SHA-256, then near-duplicate removal by MinHash over character 5-gram shingles.
- (7) Document-level, source-stratified splitting on a fixed seed.

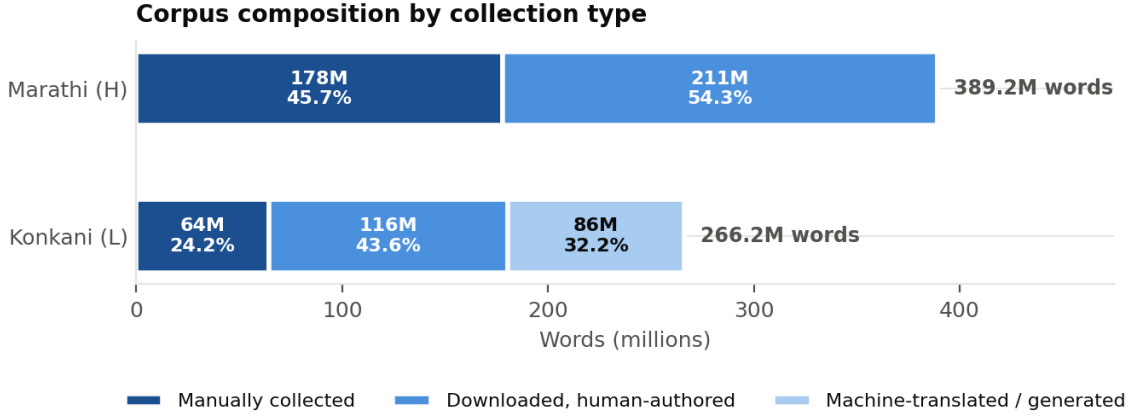
Every accepted document carries a manifest row: source, URL, collection method and type, access date, character and word counts, script profile, language-ID score, content hash. So the manual fraction can be checked document by document instead of just being claimed as a total.

Two choices there need explaining. Deduplication uses character 5-grams, not word  $n$ -grams: both corpora are OCR-heavy, and one OCR error damages five character shingles out of thousands but destroys a whole word token and every word  $n$ -gram containing it. Order matters too. Applied after packing sentences into documents it catches nothing, since no two packed documents are byte-identical; moving it to the unit level removed 197,656 repeats.

Script cannot separate the two languages, and general-purpose identifiers treat Konkani inconsistently, so the discriminator uses closed-class function words that a text can neither avoid nor borrow: *aani/aanii* (“and”), *aahe/aasaa* (“is”), *mii/haanva* (“I”). Below five total marker hits it returns *undecided* instead of guessing, and the score must clear a 0.30 margin. Of 50,000 sampled Konkani documents, 2,505 came back undecided and none was misassigned.

**Two measurements changed the corpus.**

- My first archive.org query for Konkani books returned 44 items, which supported an argument that manual Konkani collection was not viable. The query was matching a locally held spelling list instead of the archive index. Corrected to `language:kok AND mediatype:texts` it returns 5,093 items, and manual Konkani went from 1.46M words to 64.4M.
- IndicCorp v2 ships `data/gom.txt` at 533 MB labelled Goan Konkani, larger by size than every other Konkani source combined. My gate rejected 70% of it. I packed three populations to the same ~300-word length so length could not be confounded with language, and scored them identically: reference Konkani returned 94.7% `kok`, reference Marathi 100% `mr`, and `gom.txt` **83.7% `mr`**. It sits on the Marathi reference, not between the two. Taken



**Figure 1:** Corpus composition. Marathi reached target on entirely human-authored text; Konkani reached it only with 32.2% machine-translated or generated text, because real Konkani plateaued at 180.5M words.

|                                        | Marathi (H)                 | Konkani (L)             |
|----------------------------------------|-----------------------------|-------------------------|
| Documents accepted                     | 2,887,867                   | 323,112                 |
| Words accepted                         | 389,218,163                 | 266,211,363             |
| manually collected                     | 177,781,779 (45.7%)         | 64,435,242 (24.2%)      |
| downloaded, human-authored             | 211,436,384 (54.3%)         | 116,071,660 (43.6%)     |
| machine-translated / generated         | 0                           | 85,704,461 (32.2%)      |
| Train / val / test documents           | 2,058,209 / 21,001 / 21,001 | 261,186 / 2,655 / 2,655 |
| <b>Training tokens available</b>       | <b>872,024,099</b>          | <b>506,259,368</b>      |
| of which manually collected            | 475,466,104 (54.5%)         | 159,563,967 (31.5%)     |
| clears the ~500M target                | yes, 1.74×                  | yes, 1.01×              |
| Vocabulary size                        | 2,500                       | 2,500                   |
| Fertility (tokens/word, held-out)      | 2.6301                      | 2.5279                  |
| Characters per token                   | 2.6416                      | 2.5583                  |
| Whole-word token rate                  | 34.4%                       | 31.0%                   |
| Unknown-token rate                     | 0.000000%                   | 0.000000%               |
| Vocabulary utilisation                 | 92.6%                       | 94.2%                   |
| Pieces seen exactly once               | 10                          | 2                       |
| Documents leaked between splits        | 0                           | 0                       |
| Documents shared with the other corpus | 0                           | 0                       |
| Near-duplicate pairs across corpora    | 0                           | 0                       |
| Cross-language contamination           | 0.000%                      | 0.000%                  |

**Table 1:** Phase 1 corpus and tokenizer statistics. Both corpora clear the ~500M training-token target and the 20% manual floor. Konkani’s tokenizer is marginally *better* than Marathi’s on every segmentation measure, so nothing in the tokenization disadvantages the lower-resource model.

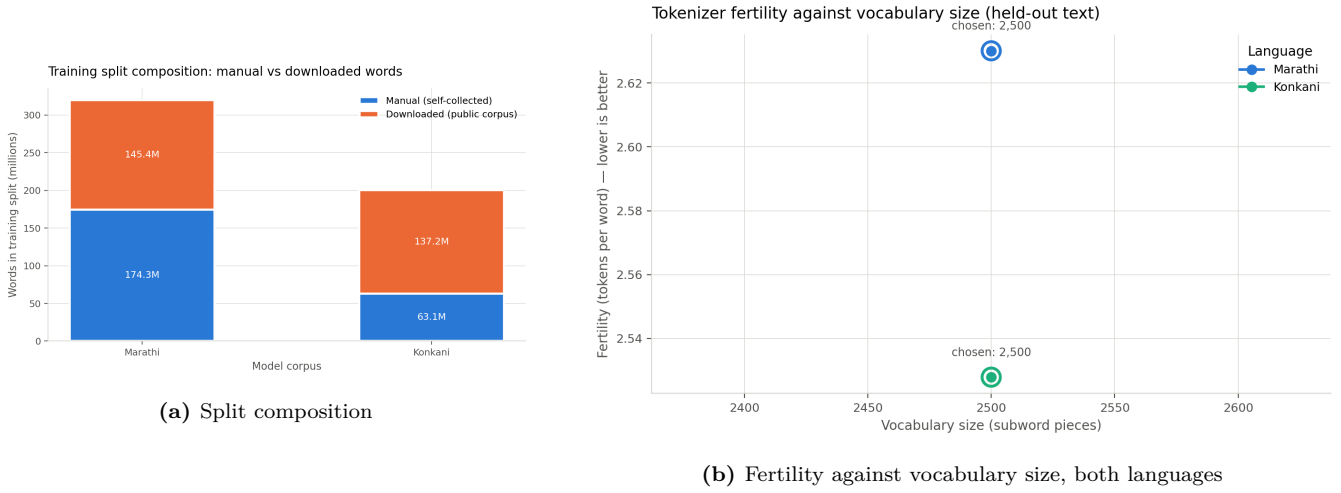
unfiltered it would have put roughly 25M words of Marathi into Model L’s corpus.

Real Konkani text ran out at 180.5M words, well short of target. There was no more of it to find, so with approval to do so I filled the gap with machine-translated and LLM-generated text. That portion is 85,704,461 words, 32.2% of the corpus, and it stays separable three ways: `CollectionType.MACHINE_TRANSLATED` is its own enum member, so `is_manual` is `False` and it can never count toward the 20% floor nor be folded into the downloaded figure; it lives in its own directory and Drive archive; and every statistics table reports it as a third bucket. The last 1.46M tokens to target came from BPCC `gom_Deva`, real human-translated Konkani, which is why the corpus crosses the target on 67.8% real data. Marathi uses no synthetic data.

Each tokenizer is SentencePiece BPE, trained on its own language’s train split only: no pretrained tokenizer, no shared vocabulary. Byte fallback adds 256 `<0xNN>` pieces and makes any Unicode string representable, which is why the unknown rate is 0.000000%. The segmentation is morphologically plausible: `__mahaaraashtra + aatiila` splits stem from locative suffix, `__gonya + chii` splits stem from genitive, and frequent Konkani function words such as `__aasaa` and `__aani` survive as single pieces.

**Vocabulary size: 2,500.** An order of magnitude below the usual range, so it needs defending.

- Byte fallback pins the unknown-token rate at 0.000000% for every candidate size, so the number that would normally settle this tells me nothing.
- Parameter cost decides instead. At  $d_{\text{model}} = 512$  with an untied head, embedding plus unembedding cost  $2Vd$ . A 10,000-piece vocabulary spends 41% of a 25M budget on two lookup tables against 10% at 2,500, which is about two Transformer layers’ worth.



**Figure 2:** (a) train, validation and test are stratified by source, so the collection mix is preserved in every split instead of concentrating in one. (b) fertility falls as the vocabulary grows in both languages at a similar rate. The unknown-token rate, which would ordinarily drive the vocabulary choice, is identically zero at every candidate size because of byte fallback.

| Hyper-parameter        | Value                        |
|------------------------|------------------------------|
| $d_{\text{model}}$     | 512                          |
| Layers                 | 7                            |
| Heads                  | 8 ( $d_{\text{head}} = 64$ ) |
| Feed-forward inner dim | 2048                         |
| Context length         | 512                          |
| Vocabulary             | 2,500                        |
| Dropout                | 0.1                          |
| Output head            | untied                       |
| <b>Parameters</b>      | <b>24,892,356</b>            |

| Training budget (each model) | Value               |
|------------------------------|---------------------|
| Optimizer steps              | 3,814               |
| Tokens per step              | 131,072             |
| <b>Tokens consumed</b>       | <b>499,908,608</b>  |
| as a share of Marathi corpus | 57.3% (0.57 epochs) |
| as a share of Konkani corpus | 98.7% (0.99 epochs) |
| Peak learning rate           | $3e-4$              |
| Wall-clock (T4)              | 3.57 h / 3.41 h     |

**Table 2:** Architecture and training budget. Both models saw exactly the same number of tokens, 499,908,608, so results differ only by the data behind those tokens. Marathi had 872.0M training tokens available and used just over half; Konkani had 506.3M and used almost all of them, a little under one full pass.

- The counter-argument is recorded: weight tying halves that cost, so the parameter argument rules out an *untied* large vocabulary, not a large vocabulary in general.
- The price is real. Konkani fertility rises from 2.1703 at 5,000 to 2.5148 at 2,500 and whole-word coverage falls from 40.6% to 30.6%, so every sequence carries about 16% more tokens for the same text. Token count is fertility  $\times$  words, so a small vocabulary also inflates the reported token total without adding data. Word counts appear alongside token counts throughout for that reason.

## 2. Phase 2: Model Implementation, Pretraining and Evaluation

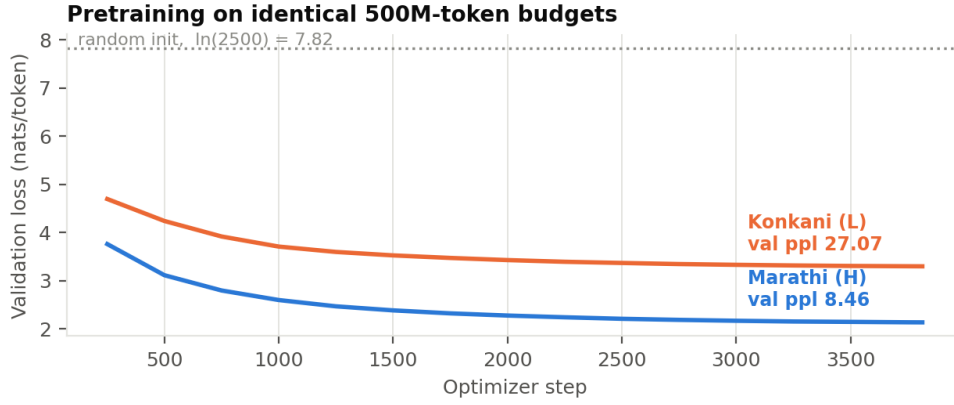
The model is a decoder-only GPT-style Transformer built only from `nn.Linear`, `nn.Embedding`, `nn.LayerNorm` and `nn.Dropout`. No `nn.Transformer*`, no library attention. Multi-head causal self-attention is implemented from primitives: separate Q/K/V projections, explicit reshape to  $(B, h, T, d_{\text{head}})$ , scaled dot-product softmax  $\text{softmax}(QK^T / \sqrt{d_{\text{head}}} + M)V$ , output projection  $W_O$ .

Three implementation choices carry weight:

- **The causal mask is additive.**  $-\infty$  goes in before the softmax, not a multiply afterwards. Zeroing after the softmax leaves rows summing to less than one.
- **Depth over width.** Inside a block, width costs  $12d^2$  while depth costs linearly, so seven layers at 512 buys more sequential composition than four at 768 for the same parameters.
- **Untied output head.** Tying saves 1,282,500 parameters, 5% of the budget at vocabulary 2,500. At vocabulary 50,000 that saving is worth having; here it is not, so it went on letting a token’s input and output representations differ.

`tools/verify_model.py` runs 18 checks, all executed before any GPU time was spent. The four that matter:

- Changing the token at position  $t + 1$  does not change the logits at position  $t$ . Tested at every position; largest difference 0.000e+00.
- Changing token 0 *does* change the last position. Without this, a model ignoring its input entirely would pass the first check.
- Attention weights above the diagonal are exactly zero, and every row sums to 1.
- Untrained loss is 7.85 against  $\ln(2500) = 7.82$ .



**Figure 3:** Pretraining on identical token budgets. Validation loss sits slightly below training loss throughout because training loss is measured per batch with dropout active. The entire gap between the two curves is attributable to the data.

| Held-out test split              | Marathi (H)          | Konkani (L)         | Ratio L/H    |
|----------------------------------|----------------------|---------------------|--------------|
| Cross-entropy (nats/token)       | 2.1544               | 3.2794              | +1.125 nats  |
| Perplexity                       | 8.62                 | 26.56               | 3.08×        |
| <b>Bits per byte</b>             | <b>0.4728</b>        | <b>0.7086</b>       | <b>1.50×</b> |
| Bytes per token                  | 6.574                | 6.677               | 1.02×        |
| BLEU-4 / chrF / ROUGE-L, greedy  | 7.35 / 25.84 / 13.14 | 0.00 / 9.80 / 3.25  |              |
| BLEU-4 / chrF / ROUGE-L, $T=0.5$ | 8.03 / 27.82 / 13.93 | 0.00 / 16.36 / 5.53 |              |
| BLEU-4 / chrF / ROUGE-L, $T=1.0$ | 3.57 / 27.34 / 9.80  | 0.00 / 20.47 / 4.47 |              |
| BLEU-4 / chrF / ROUGE-L, $T=1.5$ | 1.13 / 22.32 / 4.33  | 0.00 / 21.02 / 2.30 |              |
| Distinct-1 / Distinct-2, greedy  | 0.145 / 0.283        | 0.076 / 0.111       |              |
| 4-gram repetition, greedy        | 0.629                | 0.875               | 1.39×        |
| 4-gram repetition, $T=1.0$       | 0.047                | 0.037               | 0.79×        |

**Table 3:** Phase 2 evaluation: 24 held-out prefixes of 64 tokens, 128 tokens generated per setting, same prompts for every setting and both models.

A leaking causal mask still produces a falling loss curve and a plausible perplexity, so it cannot be caught from training alone. That is the reason for checking first.

**Pretraining.** AdamW,  $\beta = (0.9, 0.95)$ , weight decay 0.1 on matrices only. Peak learning rate  $3e-4$ , linear warmup over 2% of steps, cosine decay to 10% of peak. Effective batch 131,072 tokens ( $32 \times 512 \times 8$  gradient accumulation), gradient clipping at global norm 1.0, mixed precision with fp32 master weights. Both runs went on Kaggle, one model per T4, in parallel.

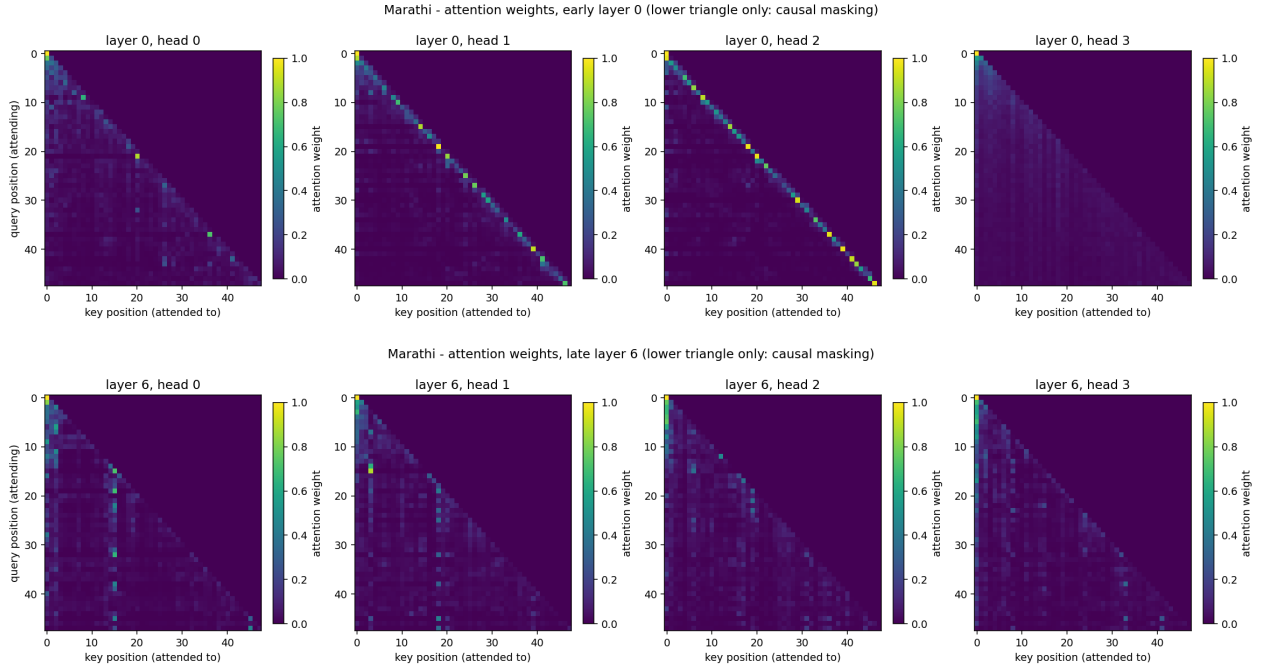
Checkpoints hold weights, optimizer state, GradScaler state, step, best validation loss and both configs, written atomically. Three post-run checks confirm the schedule ran to completion:

- Final learning rate  $3.00e-05$ , the cosine floor.
- Gradient norms 0.509–0.620 over the last five logged steps.
- No loss spikes. A resume that had lost its optimizer state would show them.

**Perplexity and bits per byte disagree, and BPB is the one to quote.** Perplexity is per token, so it is not comparable across two tokenizers; bits per byte divides the same likelihood by UTF-8 bytes. The gaps they report differ by a factor of two,  $3.08\times$  against  $1.50\times$ , for a mechanical reason: perplexity is exponential in the loss and BPB is linear, so one 1.125-nat gap becomes  $e^{1.125} = 3.08$  in one and 1.5 in the other. The figure I stand behind is the second one: Model L needs about 50% more bits per byte. These two tokenizers do happen to compress almost identically, 6.574 against 6.677 bytes per token, which is a coincidence of these corpora and not a general licence.

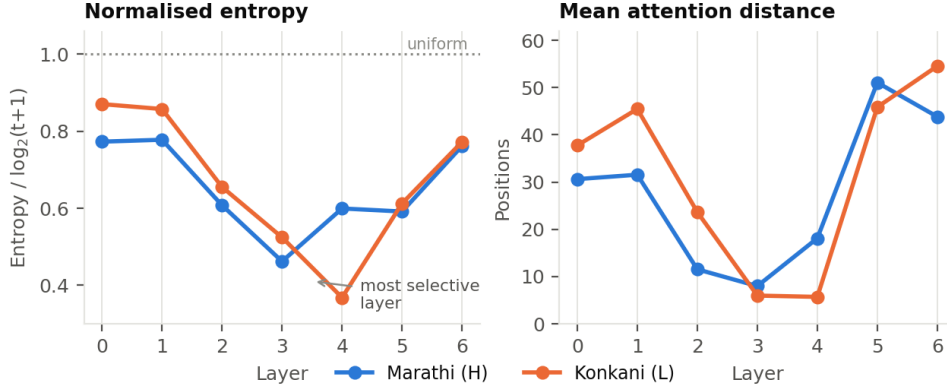
Konkani’s BLEU of exactly 0.00 needs explaining. Corpus BLEU is the geometric mean of modified  $n$ -gram precisions for  $n = 1 \dots 4$ , and Konkani scores 6.894 at unigram, 0.624 at bigram, then 0.000 at both trigram and 4-gram. Across 24 generations of 128 tokens it produced no trigram appearing in its reference continuation, and one zero order zeroes the mean. I did not smooth: smoothed sentence-BLEU returns a small positive number and conceals exactly that fact. chrF is the informative metric at this quality level. Character  $n$ -grams give partial credit for a correct stem with a different suffix, so chrF keeps moving, 9.80 to 21.02 across the four settings, where BLEU is pinned at zero. In a morphologically rich Devanagari language a fluent continuation that inflects a stem differently from the reference earns nothing under BLEU.

Perplexity also cannot see looping, since a repeated high-probability phrase scores well, so Distinct- $n$  and 4-gram repetition were measured separately. Greedy Konkani repeats 87.5% of its 4-grams with only 7.6% distinct tokens; Marathi is at 62.9% and 14.5%. Marathi at  $T = 0.5$  produces grammatical, idiomatic multi-sentence text and even reproduces the dateline convention of Marathi news, *ratnaagirii* :, before drifting. Konkani greedy opens correctly,



**Figure 4:** Marathi attention weights, first four heads, early layer 0 (top) and final layer 6 (bottom). The uniformly dark upper triangle is the causal mask drawn directly. Heads 1 and 2 at layer 0 carry a bright diagonal, the local/positional band; head 3 is almost uniform over the allowed region. By layer 6 the bright first column, the attention sink, dominates. Konkani heatmaps have the same shape and are in `report/figures/`.

#### Attention is U-shaped in both models, not early-local / late-global



**Figure 5:** Normalised attention entropy and mean attention distance by layer. Both models contradict the expected early-local / late-global pattern in the same way.

then locks onto *... baarbaraa haangaachyaa baarbaraa haangaachyaa...* and never reopens. Both are locally fluent and globally incoherent. That is what 25M parameters on 500M tokens buys.

The per-head statistics come from 32 held-out sequences of 256 tokens, with entropy normalised by  $\log_2(t+1)$ , the causal maximum at query position  $t$ , so heads at different positions are comparable. I expected low-entropy local heads early and high-entropy long-range heads late. Neither model does that. Both are U-shaped:

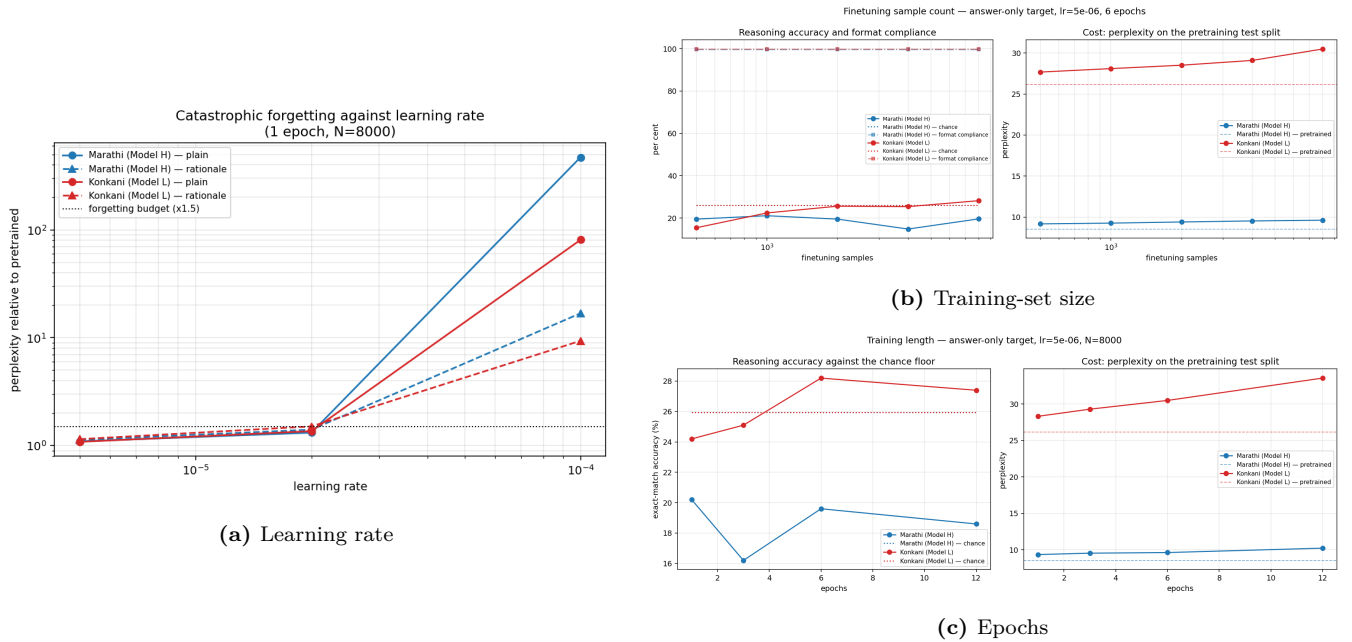
- Layers 0–1 are diffuse and moderately long-range, normalised entropy 0.773 and 0.870, closer to broadcasting context than selecting from it.
- Layers 3–4 are focused and local, 0.462 and 0.524, mean distance 7.95 and 5.94 positions.
- Layers 5–6 go diffuse and long-range again.

The selective work happens mid-stack, and both models show it independently on different corpora, so the effect looks architecture-driven, not language-specific. Heads within a layer differ sharply too: Konkani layer 2 holds one head at mean distance 4.13 and another at 105.61, a 25 $\times$  spread. Had the heads collapsed onto one behaviour, a single 512-dimensional head would have done the same job.

### 3. Phase 3: Reasoning Finetuning and Attention Analysis

Five task families per language: `compare_two` (2 entities), `equality` (2), `superlative_three` (3), `transitive_2hop` (3), `transitive_3hop` (4). 8,000 training items,  $\sim$ 490 validation and 1,000 test items per language. A `transitive_2hop` item reads *prashna: priyaache vajana giitaapekshaa jaasta aahe. giitaache vajana prakaashaapekshaa jaasta aahe. sarvaata jaasta vajana konaache aahe?* with target *? uttara: priyaa*.

Four design properties keep the benchmark honest:



**Figure 6:** The three calibration sweeps. (a) the  $\times 1.5$  forgetting budget was fixed before any accuracy number was inspected, and  $1e-4$  is past the cliff by three orders of magnitude for Marathi. (b) at six epochs accuracy does not improve with more data for Marathi while perplexity keeps degrading. (c) both languages peak at six epochs and turn over at twelve while perplexity keeps climbing.

- Items are generated programmatically, and ground truth is computed from the sampled values, never asserted.
- Entity pools are **disjoint** between train and test: 16 train names against 6 test for Marathi, 24 against 8 for Konkani.
- Three surface patterns express the same relations differently, and one, **p\_less**, is **held out of training entirely**. It appears only at test time, which separates “learned these templates” from “learned the relation”.
- Every name, genitive form, oblique stem and numeral convention was verified for attestation against a 400 MB sample of that language’s own training split, so a wrong answer cannot be blamed on an unseen word.

Numerals use Devanagari digits and comparisons use language words. The symbols  $>$  and  $<$  fall outside a 2,500-piece Devanagari vocabulary and would arrive as byte pieces. Leakage checks returned zero entity and zero prompt overlap, byte-fallback rate is 0.000000%, and the Konkani set scores 8,000/8,000 **kok** under the Phase 1 discriminator.

**Protocol.** Supervised finetuning with a prompt-masked loss. Target positions covering the prompt are set to  $-100$  so **F.cross\_entropy** ignores them, and loss is computed over the answer span alone. The mask begins at **n\_prompt - 1**, not **n\_prompt**: the first answer token is predicted from the last prompt token. Each model is finetuned from its own pretrained checkpoint, and no weights, data or tokenizer cross between the two. Evaluation is greedy generation bucketed by exact prompt length, so no padding is needed; with learned absolute positional embeddings, left-padding would shift every position.

**The first attempt failed, and the failure set the method.** My first sweep varied sample count while holding the learning rate at  $1e-4$  for three epochs. It overwrote the pretrained model at every sample count, with Marathi perplexity rising from 8.55 to 22,126, and accuracy sat at 14.10% identically to two decimal places across all five sizes. Five points on that curve were five measurements of the same collapse.

The corrected notebook calibrates first, on a learning-rate  $\times$  training-target grid at one epoch, with a forgetting budget of  $\times 1.5$  fixed *before* any accuracy number was inspected. Configurations outside the budget were treated as unusable regardless of score. A reasoning result taken from a model whose perplexity rose a hundredfold is not a result about reasoning. The chosen recipe: answer-only targets, lr  $5e-6$ , six epochs,  $N = 8,000$ .

One floor is not enough here. Uniform chance is  $1/(k + 1)$  per item and  $k$  varies by family, so the significance test sums per-item variance  $\sum p_i(1 - p_i)$  instead of assuming one  $p$  across the set. But a model that ignores the question and always emits the most common answer scores the *majority-class* rate: 31.10% for Marathi, 23.30% for Konkani. Both floors are reported everywhere, and the majority-class floor is the stricter one for Model H.

**Finetuning taught both models the output format completely and the task not at all.** Compliance went from nothing to almost perfect, and every generation is exactly ? **<marker>**: **<one name>**: correct marker, correct spacing, a single plausible entity, EOS in the right place. Accuracy on the four families that require comparing entities is 6.41% and 16.90% against a 24.72% chance floor. Both sit far below guessing.

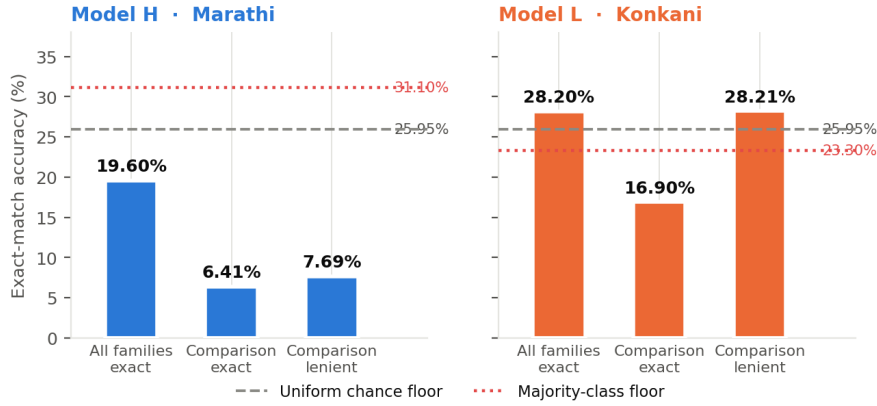
Model L’s overall 28.20% clears uniform chance at  $p = 0.051$ , but that margin is entirely the **equality** family, which has one gold label in Marathi and two in Konkani with one covering 69%. One fifth of the training set is solvable by a constant, and that constant is the most-rewarded token in the set. The flaw is in data I generated myself, and I report it as mine.

The two models fail in opposite ways:

|                                                    | Marathi (H)                                | Konkani (L)                                  |
|----------------------------------------------------|--------------------------------------------|----------------------------------------------|
| Pretrained: format compliance / accuracy           | 0.00% / 0.00%                              | 0.00% / 0.00%                                |
| Finetuned: format compliance                       | 99.60%                                     | 99.90%                                       |
| Finetuned: accuracy, all families                  | 19.60%                                     | 28.20%                                       |
| Pretraining perplexity, before $\rightarrow$ after | 8.55 $\rightarrow$ 9.72 ( $\times 1.137$ ) | 26.14 $\rightarrow$ 30.85 ( $\times 1.180$ ) |
| Uniform chance floor                               | 25.95%                                     | 25.95%                                       |
| Majority-class floor                               | 31.10%                                     | 23.30%                                       |
| Accuracy excluding equality                        | <b>6.41%</b> ( $z = -12.50$ )              | <b>16.90%</b> ( $z = -5.34$ )                |
| The same, scored leniently                         | 7.69% ( $z = -11.63$ )                     | <b>28.21%</b> ( $z = +2.38$ , $p = 0.009$ )  |
| Uniform chance, same subset                        | 24.72%                                     | 24.72%                                       |
| Held-out surface pattern <b>p_less</b>             | 0.00%                                      | 4.55%                                        |
| Trained pattern, same relation inverted            | 18.18%                                     | 34.62%                                       |

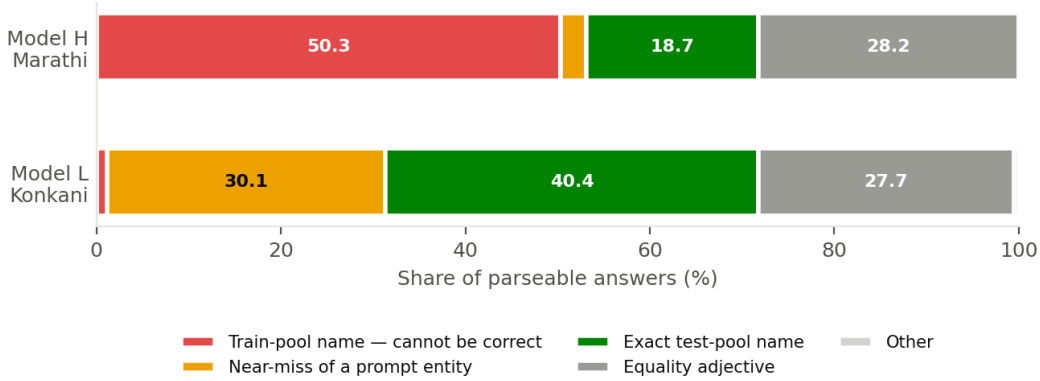
**Table 4:** Phase 3 results. The bottom block is the sharpest evidence here. On a surface pattern never seen during training both models collapse. What was learned is template-shaped, not relation-shaped.

**Only Model L clears chance, and only under lenient scoring**



**Figure 7:** Accuracy against both floors. Model L’s headline 28.20% clears uniform chance at  $p = 0.051$ , but that margin is carried entirely by the degenerate **equality** family; removing it drops Model L to 16.90% and Model H to 6.41%.

**What the wrong answers are actually made of**

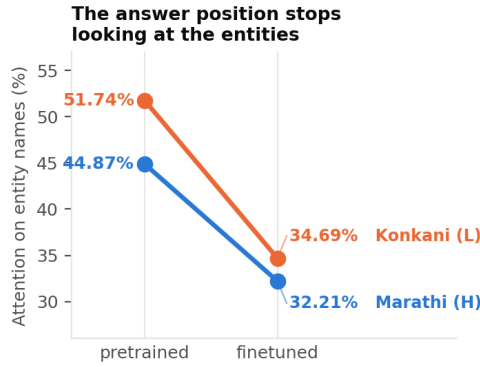


**Figure 8:** Every parseable prediction sorted by which entity pool its name could have come from. The pools are disjoint by construction, so a train-pool answer cannot be correct on any test item. The two models fail in opposite ways, and neither failure is visible in an accuracy number.

- **Model H is not reading the prompt.** 50.3% of its answers are names appearing only in training: *raama* 273 times, *raahula* 148, *lataa* 67. None can be correct on any test item, because no test prompt contains them. It learned which names tend to be answers and recites them.
- **Model L is reading the prompt and failing to spell the answer.** It hallucinates train-pool names only 1.2% of the time. Instead 30.1% of its answers are near-misses of an entity genuinely present (*miira* for *miiraa* 185 times, *krishnaa* for *krishna* 84), and a further 40.4% are exact test-pool names. So 70.5% of its answers refer to an entity actually in the question it was asked.

That second case is a generation failure sitting on top of a partial success, and exact match scores it identically to naming the wrong entity.

**A lenient score, declared as post-hoc.** To separate “does not know the answer” from “cannot spell it”, a prediction was also scored correct if it is a prefix or extension of the gold *and of no other candidate*; the uniqueness



**Figure 9:** Share of the answer position’s attention falling on the entity names, before and after finetuning, averaged over 64 held-out prompts and all eight heads of the final layer. Both models attend less to the entities after finetuning, with the mass moving to the question clause.

clause stops the rule manufacturing accuracy out of ambiguity. Model H moves 6.41% to 7.69% and stays far below chance. Model L moves 16.90% to 28.21%, from  $z = -5.34$  to  $z = +2.38$  ( $p = 0.009$ ), with `superlative_three`, `transitive_2hop` and `transitive_3hop` each clearing their own floor independently. This metric was defined after seeing the error distribution. D-060 records that plainly, exact match remains the headline figure in every table, and the matching rule is stated in full so it can be checked. The negative conclusion survives for Model H and does not survive for Model L.

The surface you measure on changes the answer. Over the pretraining test split, meaning news, circulars and OCR’d books, finetuning moved mean entropy over all 56 heads by +0.050 bits for Marathi and +0.001 for Konkani. Essentially nothing. But reasoning prompts are a tiny, highly structured slice of input space that the corpus contains nothing shaped like, so a change confined to them is invisible in that average.

Re-measured on 64 held-out `transitive_3hop` prompts, the picture inverts:

- Attention sharpens by  $-0.171$  bits for Marathi and  $-0.219$  for Konkani, in *every single layer* of both models.
- The largest movement is mid-stack; Konkani layer 3 falls 0.662 bits, its mean attention distance from 4.71 to 3.74 positions.
- The last prompt token is the position whose output becomes the first answer token, so its attention row decides the answer. Bucketed by what each prompt token means, both models attend *less* to the entity names after finetuning: 44.87% to 32.21% for Marathi, 51.74% to 34.69% for Konkani, with the mass moving to the question clause.

The tool is validated by running it with the same checkpoint in both arms, which produces deltas of exactly +0.000. Heatmaps of the same layer on a reasoning prompt are in `report/figures/` under `phase3_attn_reasoning_*`.

This corroborates the error taxonomy from the opposite direction: the taxonomy found in the outputs that Model H had stopped reading the prompt, and the attention analysis finds in the weights that the answer-producing position attends 12.7 points less to the prompt’s entities. Model L needs a more careful reading. It also attends less, but 34.69% is still a third of the budget, and 70.5% of its answers do refer to an entity present in the prompt. A third of the budget is enough to identify which entities are in play. What Model L fails at is emitting the exact token sequence for the one it chose.

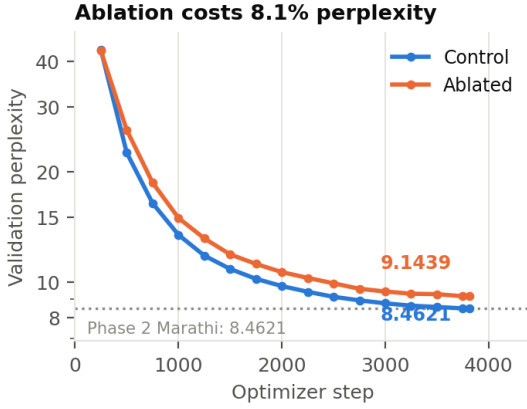
## 4. Required Questions

**1. How did data scale and quality differ between H and L?** They differ in kind, not only in scale.

- Marathi: 389.2M words, 872.0M training tokens, entirely human-authored, 45.7% manually collected.
- Konkani: 266.2M words, 506.3M tokens, but only because 32.2% is machine-translated or generated. Real Konkani plateaued at 180.5M words once the source repositories were exhausted.
- Manual share of training tokens: 54.5% against 31.5%.
- Both models then trained on exactly 499,908,608 tokens, so Marathi used 57.3% of its corpus and Konkani 98.7% of its own. Model L saw nearly one full pass, Model H barely half.
- Quality differences are invisible in the tokenizer statistics but very visible in provenance: Konkani needed ten downloaded sources against Marathi’s one, three of those ten did not match their labels when measured, and the largest Konkani file on offer was 83.7% Marathi.

**2. How do the language-modelling and reasoning results compare across resource tiers?** They point in opposite directions. On intrinsic language modelling the gap behaves as expected: Model L needs 50% more bits per byte, 0.7086 against 0.4728, its BLEU is zero against 8.03, and it repeats 87.5% of its 4-grams greedily against 62.9%. On reasoning the ordering reverses. Model H scores 6.41% on the comparison families and Model L 16.90%, and under lenient scoring Model L is the only one of the two to clear its chance floor at all ( $z = +2.38$ ,  $p = 0.009$ ) while Model H sits at  $z = -11.63$ . The better language model is the worse reasoner here. Catastrophic forgetting was comparable and small in both,  $\times 1.14$  and  $\times 1.18$ , so it does not explain the reversal.

**3. What tokenizer or corpus factors most affected the lower-resource model?** The tokenizer can be ruled out directly. Both vocabularies are 2,500 with byte fallback and a 0.000000% unknown rate; Konkani’s fertility is



|                             | Control    | Ablated    |
|-----------------------------|------------|------------|
| Positional embeddings       | present    | removed    |
| Parameters                  | 24,892,356 | 24,630,212 |
| Final validation perplexity | 8.4621     | 9.1439     |
| Test perplexity             | 9.0846     | 9.8451     |
| Test bits per byte          | 0.4767     | 0.4941     |
| 4-gram repetition, greedy   | 0.672      | 0.723      |
| 4-gram repetition, $T=0.5$  | 0.412      | 0.515      |
| 4-gram repetition, $T=1.0$  | 0.098      | 0.101      |

**Figure 10:** The two arms separate by step 500 and never re-converge. Removing 262,144 parameters, 1.05% of the model, costs 8.1% perplexity.

marginally better than Marathi’s, 2.5279 against 2.6301 tokens per word, its vocabulary utilisation is higher at 94.2% against 92.6%, and it has fewer hapax pieces, 2 against 10. Nothing in the segmentation disadvantages Model L. The corpus is where the difference lives. A third of Konkani’s training text is synthetic, the real portion came from ten heterogeneous sources of varying reliability, and an 87.5% greedy repetition rate is the signature of a model that has seen less genuine variety, not one that has been segmented badly. I give this as the reading the evidence supports, *not* as a proven cause. The ablation that would settle it is retraining Model L on its 67.8% real text alone, and I did not run it.

**4. What evidence explains the observed differences?** Per-example inspection, not aggregate scores. Two independent instruments agree, and two further checks support the negative headline:

- The error taxonomy separates the models cleanly: 50.3% train-pool-only hallucination for Model H against 1.2% for Model L, whose errors are instead 30.1% near-misses of entities genuinely present.
- The attention measurement finds the answer-producing position attending 12.7 and 17.1 points less to the entity names after finetuning, which is the mechanism behind what the taxonomy sees in the outputs.
- The held-out surface pattern `p_less`, excluded since the data was designed, scores 0.00% and 4.55% against 18.18% and 34.62% on the trained pattern expressing the same relation inverted.
- Pretrained baselines score 0.000% everywhere, so nothing reported here is inherited from pretraining.

## 5. Bonus: Retraining Without Positional Embeddings

On a separate `bonus-no-positional` branch built on top of the submitted Phase 3, Marathi was pretrained twice from scratch, side by side on one T4 each, same seed, same data, same 500M-token budget, differing only in the learned absolute positional embedding table.

Removing it sets `position_embedding` to `None`; it is not zeroed. A zeroed embedding still receives gradient and would relearn itself within a few hundred steps, so the run would measure nothing while appearing to work. The parameter count drops by exactly  $262,144 = 512 \times 512$ , verified by a parametrised test, and the flag defaults to off so no Phase 1–3 deliverable is affected. The control is a fresh retrain, not the Phase 2 checkpoint, so both arms share a driver, a GPU and a wall clock. It reproduced Phase 2 exactly, validation perplexity 8.4621 against 8.4621 to four decimal places, which also shows the Phase 2 run is reproducible across sessions weeks apart.

B-004, written before the run, predicted a cost of  $\times 1.5$  to  $\times 3$ , on the grounds that self-attention is permutation-invariant. Measured cost:  $\times 1.081$ , which fired that entry’s own falsification condition. Causal masking leaks position by itself, since position  $t$  attends to exactly  $t + 1$  tokens and that count is a positional signal. On an untrained ablated model, permuting a 16-token input already moves the last position’s logits by 1.15.

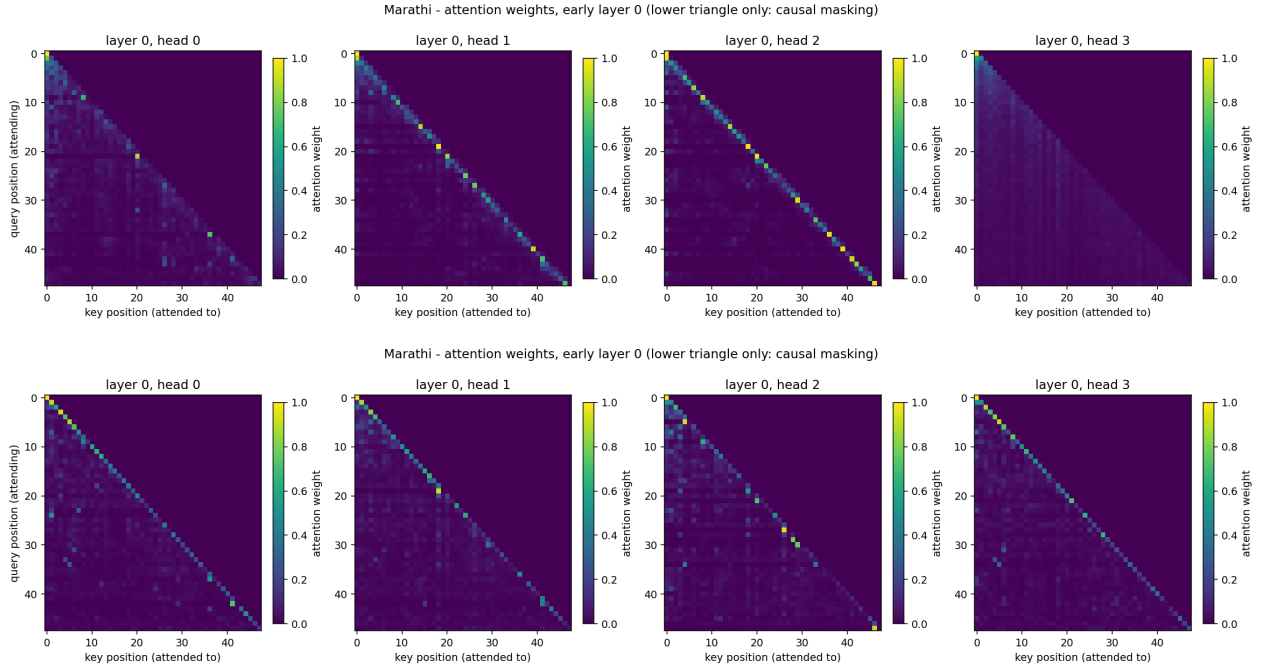
What breaks shows up in the attention, not the headline number:

- Layers 0–2 become far more diffuse, entropy +0.74, +0.89 and +1.75 bits with mean distance roughly tripling. An early head cannot form a local window when the representation does not yet encode position.
- Layers 4 and 5 become markedly more local. Layer 4 is the only layer in either model whose entropy falls, by 0.70 bits, with mean distance 18.1 to 11.1.
- Behaviourally, what degrades is tracking position within a span. 4-gram repetition rises 0.672 to 0.723 greedily and 0.412 to 0.515 at  $T = 0.5$ , and the difference vanishes at  $T = 1.0$  where sampling noise dominates.

The ablated model reconstructs locality mid-network instead of receiving it at the input, and pays for that in depth. In a seven-layer model, that is what an 8% perplexity gap looks like.

## 6. Limitations

- (1) The `equality` family is degenerate, and that is my fault: one fifth of the Phase 3 training set is solvable by a constant. The finding landed two days before the deadline, and regenerating would have invalidated every run reported here, so it is recorded instead of fixed. Every headline figure is also given with the family removed.



**Figure 11:** Layer 0 with positional embeddings (top) and without (bottom). The control has formed the local diagonal band visible in heads 1 and 2; the ablated model has not, and its early heads spread over the whole allowed region. The uniformly dark upper triangle in both is the causal mask, which is also the residual positional signal that keeps the ablation as cheap as it is.

- (2) No fluent speaker reviewed the generated Konkani sentences. Every lexical item, genitive form, oblique stem and numeral convention was verified against a 400 MB sample of the language’s own training corpus, which is stronger evidence of attestation than hand-checking would give. But attestation is not a native reader judging that the sentences read naturally, and the author is not a fluent Konkani speaker.
- (3) At  $n = 1,000$  a lift must reach about 2.7 points for  $p < 0.05$ , and the largest observed is 2.25. The conclusions here concern far larger effects, but a genuinely marginal positive result could not have been established.
- (4) The reasoning-prompt attention analysis uses one family, **transitive\_3hop**. The direction of the effect is unlikely to be family-specific, being large and identical in sign across two independently finetuned models, but its magnitude may be.
- (5) The bonus covers Marathi only, so whether the ablation hurts more in the lower-resource setting is untested.
- (6) The task selects among options visible in the prompt, the easiest form of comparative reasoning. Neither model could do even that, and harder variants were not attempted.
- (7) The Phase 3 reversal between the two languages has a supported reading and no confirmed cause. I leave it open instead of filling it in with an untested explanation.

## 7. Where Automation Substituted for Human Judgment

Across all three phases thresholds were set numerically, not by reading text:

- The 70% Devanagari script gate, the 25-word length gate, the five-marker and 0.30-margin language-ID thresholds and the 0.85 near-duplicate cutoff were chosen as parameters and validated against populations of known language, not by a human inspecting kept and dropped samples.
- The vocabulary size came from a numeric selection rule, the smallest vocabulary within 2% fertility of the best candidate, not from judging tokenized output by eye. The unknown-token rate that would ordinarily inform that choice was uninformative under byte fallback.
- Attention-head labels such as “local” and “long-range” rest on entropy and mean-distance proxies plus visual heatmap reading, not on linguistically grounded probing of what a head tracks.
- The Phase 3 lexicon was checked for corpus attestation programmatically across all generated items. Naturalness was not assessed by a speaker at all.

Twice, though, that automation caught what a human reading would have missed, which is the argument for it. The language discriminator, calibrated against reference populations instead of trusted on its raw output, exposed a 533 MB file labelled Konkani as 83.7% Marathi. And testing every headline accuracy against both floors, not judging by eye, revealed that Model L’s apparent 28.20% success rested entirely on one degenerate family.

Reproduction steps, Drive links, per-phase reports, decision logs, all 44 figures and every script: [README.md](#) and [report/](#) on **phase-3**; bonus on **bonus-no-positional**.